

تعریف ساده

وقتی به کلاس به **Type Code** داره که باعث ایجاد switch یا if/else های تکراری توی متدهاش میشه، برای هر نوع به زیرکلاس بساز و رفتار متفاوت رو به زیرکلاس ها منتقل کن.

مشکل

یه کلاس داری که یه فیلد "نوع" داره (مثلاً int یا enum)، و توی متدهاش پر از switch یا if/else هست که بر اساس نوع تصمیم می گیرن.

مثال قبل از ریفکتور

csharp

```
public class Employee
{
    public const int ENGINEER = 0;
    public const int SALESMAN = 1;
    public const int MANAGER = 2;

    public int Type { get; set; }
    public string Name { get; set; }
    public decimal Salary { get; set; }

    public decimal GetBonus()
    {
        switch (Type)
        {
            case ENGINEER: return Salary * 0.1m;
            case SALESMAN: return Salary * 0.15m;
            case MANAGER: return Salary * 0.2m;
            default: throw new Exception("نوع نامعتبر");
        }
    }

    public string GetRole()
    {
        switch (Type)
        {
```

```

        ; "مهندس" case ENGINEER: return
        ; "فروشنده" case SALESMAN: return
        ; "مدیر" case MANAGER: return
        ; ("نوع نامعتبر") default: throw new Exception
    }
    {
    {
    {

```

Replace Type Code with Subclasses بعد از

csharp

```

// کلاس والد (abstract)
public abstract class Employee
{
    public string Name { get; set; }
    public decimal Salary { get; set; }

    ;()public abstract decimal GetBonus
    ;()public abstract string GetRole
}

// زیرکلاسها
public class Engineer : Employee
{
    ;public override decimal GetBonus() => Salary * 0.1m
    ; "مهندس" <= ()public override string GetRole
}

public class Salesman : Employee
{
    ;public override decimal GetBonus() => Salary * 0.15m
    ; "فروشنده" <= ()public override string GetRole
}

public class Manager : Employee
{
    ;public override decimal GetBonus() => Salary * 0.2m
    ; "مدیر" <= ()public override string GetRole
}

```



مزیت

توضیح

حذف switch	دیگه خبری از switch های تکراری نیست
هر زیرکلاس متمرکز	منطق هر نوع فقط توی یه کلاسه
اضافه کردن نوع جدید آسونه	یه زیرکلاس جدید بساز، بدون دست زدن به بقیه کد
Open/Closed Principle	کد برای اضافه شدن باز، برای تغییر بسته

⚠ When Not to Use (از صفحه)

اگه مقدار Type Code توی زمان اجرا عوض میشه (مثلاً کارمند از مهندس به مدیر ارتقا پیدا می‌کنه)، از این تکنیک استفاده نکن. چون زیرکلاس یه شیء رو نمیشه بعد از ساخته شدن تغییر داد. توی این شرایط برو سراغ **Replace Type Code with State/Strategy**.

📝 تمرین

این کد رو با **Replace Type Code with Subclasses** ریفکتور کن:

csharp

```
public class Vehicle
{
    ;public const int CAR = 0
    ;public const int TRUCK = 1
    ;public const int MOTORCYCLE = 2

    public int Type { get; set; }
    public string Model { get; set; }

    ()public decimal GetTollPrice
    }

    switch (Type)
```

```

    }

    ;case CAR: return 5000
    ;case TRUCK: return 15000
    ;case MOTORCYCLE: return 2000
    ;("نوع نامعتبر")default: throw new Exception
    {
        {

        ()public int GetMaxSpeed
        }

        switch (Type)
        }

        ;case CAR: return 180
        ;case TRUCK: return 100
        ;case MOTORCYCLE: return 220
        ;("نوع نامعتبر")default: throw new Exception
        {
            {
                {

```

راهنمایی:

- abstract → Vehicle
- متدها → abstract
- سه زیرکلاس: Car , Truck , Motorcycle

بفرست تا بررسی کنیم! 📩